

SOC Practical Task Report on Network Traffic Analysis

Name: B.PRANEETH

Date: 08-05-2026

Introduction:

During this practical task, network traffic analysis was performed using Wireshark in Kali Linux. Different types of packets such as ICMP, TCP, DNS, and ARP were captured and analyzed. Elasticsearch and Kibana services were also started successfully to understand basic monitoring and logging activities used in SOC environments.

The main purpose of this task was to understand how network packets travel in a network and how SOC analysts monitor and inspect traffic for analysis.

Objective:

- To capture live network packets using Wireshark
- To analyze different network protocols
- To understand packet filtering techniques
- To work with Elasticsearch and Kibana
- To gain practical knowledge of SOC monitoring

Tools Used:

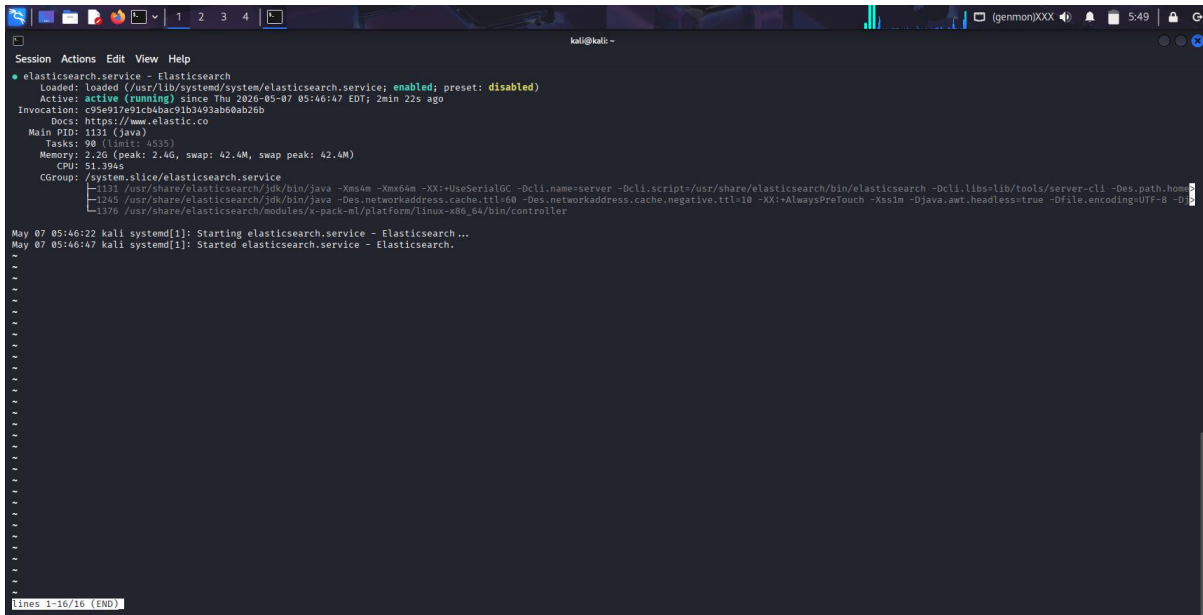
- Kali Linux
- Wireshark
- Elasticsearch
- Kibana
- VMware Workstation

Practical Implementation:

Step 1: Starting Elasticsearch Service:

The Elasticsearch service was started in Kali Linux terminal using systemctl commands. The service status was checked to verify whether the service was running properly. The service started without any errors and system resources were allocated properly. This confirms that the backend service required for Elastic Stack monitoring is functioning correctly.

Figure 1: Elasticsearch Service Status:



```
Session Actions Edit View Help
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-05-07 05:46:47 EDT; 2min 22s ago
     Invocation: c95e917e91cb4b91b1b691bb0ab2b5
       Docs: https://www.elastic.co
    Main PID: 1131 (java)
      Tasks: 90 (limit: 65535)
     Memory: 2.2G (peak: 2.4G, swap: 42.4M, swap peak: 42.4M)
        CPU: 51.394s
    CGroup: /system.slice/elasticsearch.service
            └─1131 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.nameserver -Dcli.script=/usr/share/elasticsearch/bin/elasticsearch -Dcli.lib=lib/tools/server-cli -Des.path.home
            └─1245 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10 -XX:+AlwaysPreTouch -Xss1m -Djava.awt.headless=true -Dfile.encoding=UTF-8 -D
            └─1376 /usr/share/elasticsearch/modules/x-pack-el/platform/linux-x86_64/bin/controller

May 07 05:46:22 kali systemd[1]: Starting elasticsearch.service - Elasticsearch...
May 07 05:46:47 kali systemd[1]: Started elasticsearch.service - Elasticsearch.

lines 1-16/16 (END)
```

Explanation:

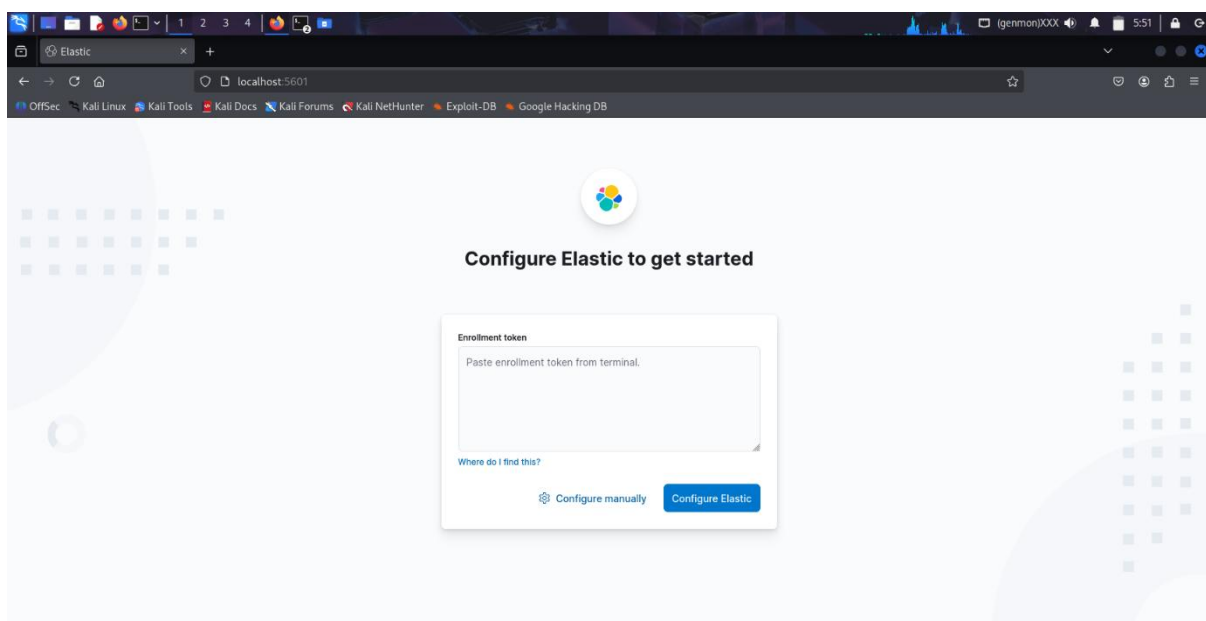
The screenshot shows that the Elasticsearch service is active and running successfully in the Kali Linux environment.

This service is important for storing and managing log data in the Elastic Stack environment.

Step 2: Opening Kibana / Elastic Interface:

After starting Elasticsearch, the Kibana interface was opened in the Firefox browser using localhost on port 5601. The web interface loaded properly in the browser and the Elastic dashboard was accessible. This step confirmed that Kibana was connected successfully with Elasticsearch services.

Figure 2: Elastic Configuration Interface:



Explanation:

The Elastic setup page was displayed successfully, confirming that the Kibana web interface was accessible. Kibana provides visualization and monitoring features which are useful for SOC analysis and log monitoring.

The screenshot shows live network packets being captured successfully from the selected interface. Different protocols and communication packets were visible during the live capture process.



Step 4: Performing Ping Test:

A ping command was executed in the terminal to generate ICMP traffic. The response time of packets was displayed successfully for each request. No packet loss was observed during the communication with the remote server.

Command Used:

```
ping google.com
```

Figure 4: Ping Command Output:

```
kali@kali: ~  
$ ping google.com  
PING google.com (142.250.182.78) 56(84) bytes of data:  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=1 ttl=128 time=34.4 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=2 ttl=128 time=37.1 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=3 ttl=128 time=30.4 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=4 ttl=128 time=40.7 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=5 ttl=128 time=55.1 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=6 ttl=128 time=50.4 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=7 ttl=128 time=42.4 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=8 ttl=128 time=26.3 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=9 ttl=128 time=29.9 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=10 ttl=128 time=23.6 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=11 ttl=128 time=36.9 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=12 ttl=128 time=65.3 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=13 ttl=128 time=35.1 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=14 ttl=128 time=31.4 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=15 ttl=128 time=46.8 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=16 ttl=128 time=57.7 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=17 ttl=128 time=40.5 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=18 ttl=128 time=30.4 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=19 ttl=128 time=41.1 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=20 ttl=128 time=51.3 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=21 ttl=128 time=30.9 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=22 ttl=128 time=41.7 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=23 ttl=128 time=95.9 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=24 ttl=128 time=26.2 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=25 ttl=128 time=83.1 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=26 ttl=128 time=60.3 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=27 ttl=128 time=40.6 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=28 ttl=128 time=32.7 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=29 ttl=128 time=76.8 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=30 ttl=128 time=32.6 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=31 ttl=128 time=47.6 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=32 ttl=128 time=29.7 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=33 ttl=128 time=50.1 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=34 ttl=128 time=40.4 ms  
64 bytes from maa0520-in-f14.1e100.net (142.250.182.78): icmp_seq=35 ttl=128 time=57.4 ms  
^C  
-- google.com ping statistics --  
35 packets transmitted, 35 received, 0% packet loss, time 34058ms  
rtt min/avg/max/mdev = 23.578/45.184/95.928/16.230 ms  
kali@kali: ~
```

Explanation:

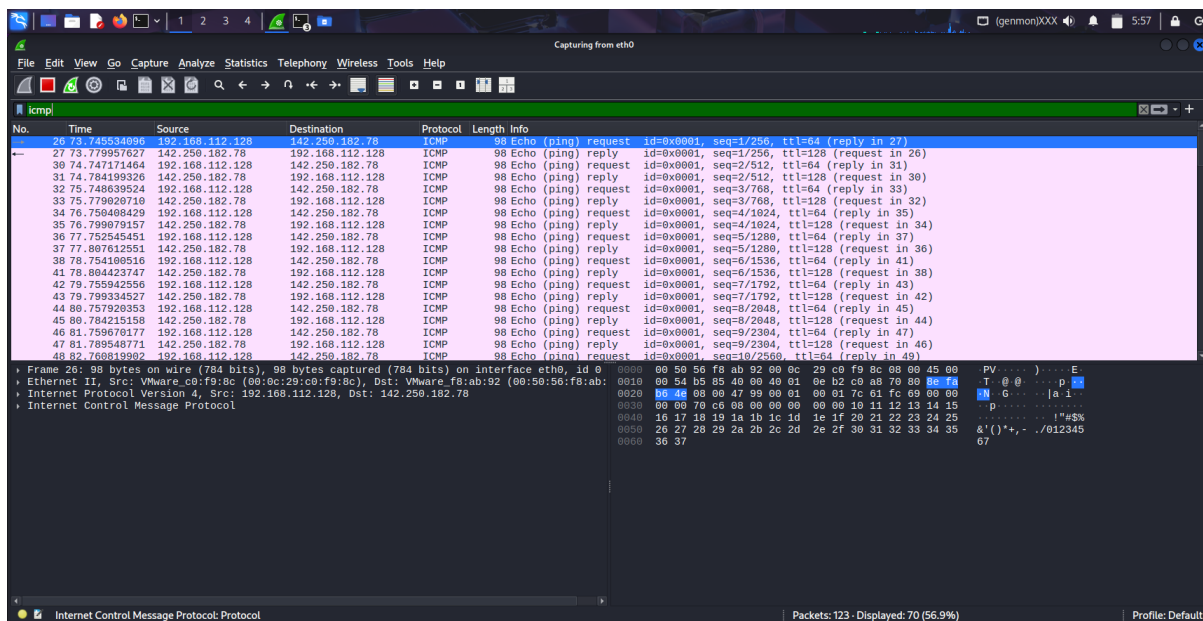
The ping command successfully sent and received packets from Google servers without packet loss. This test confirmed stable internet connectivity and generated ICMP traffic for analysis.



Step 5: ICMP Packet Analysis:

The ICMP filter was applied in Wireshark to analyze ping request and reply packets. Both request and reply packets were visible in the packet capture window. The ICMP filter helped in identifying connectivity-related traffic clearly.

Figure 5: ICMP Packet Capture:



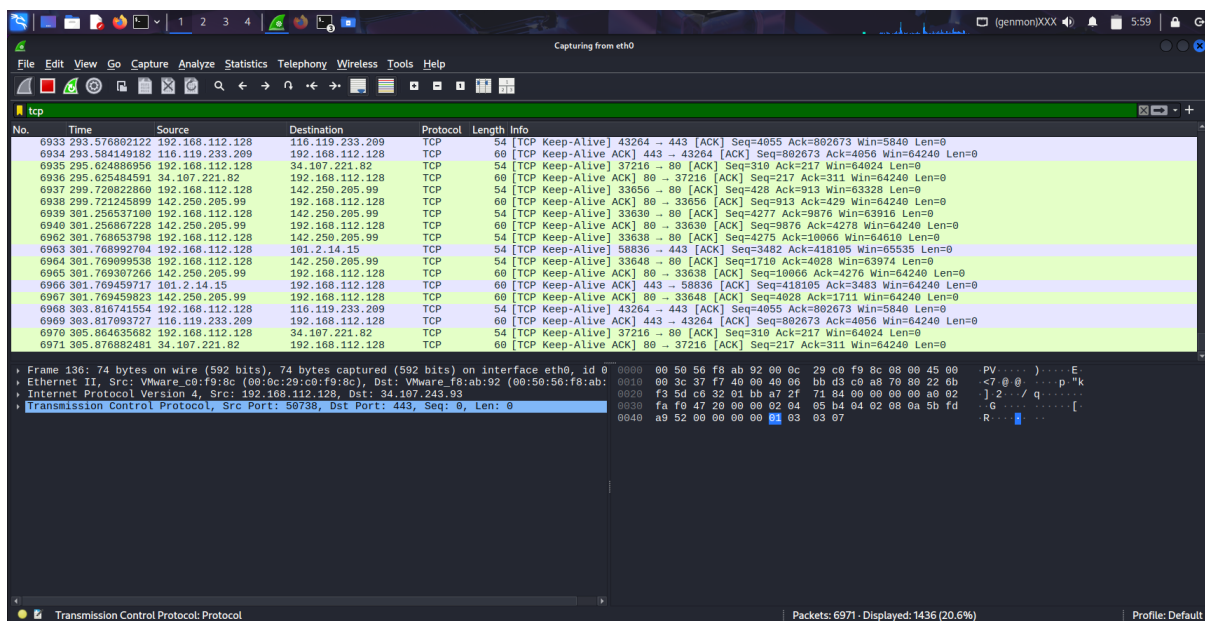
Explanation:

The captured ICMP packets displayed Echo Request and Echo Reply messages generated from the ping command. ICMP packets are commonly used for connectivity testing and troubleshooting network communication issues.

Step 6: TCP Packet Analysis:

The TCP filter was applied to monitor TCP communication packets. TCP communication packets showed reliable data transfer between source and destination systems. Different acknowledgment packets were visible during the analysis process.

Figure 6: TCP Packet Capture:



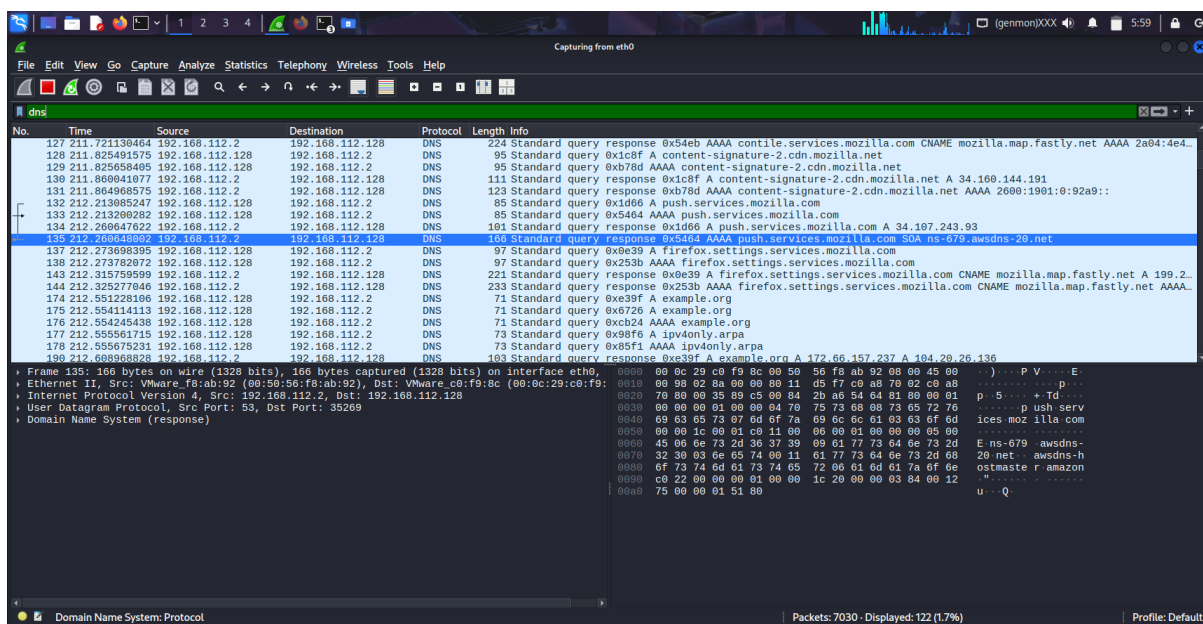
Explanation:

TCP packets such as acknowledgements and keep-alive packets were observed during network communication. TCP ensures reliable communication between systems during data transmission.

Step 7: DNS Packet Analysis:

The DNS filter was applied in Wireshark to capture DNS queries and responses. DNS queries were generated automatically while accessing websites and online services. The responses contained resolved IP addresses for requested domain names.

Figure 7: DNS Packet Capture:



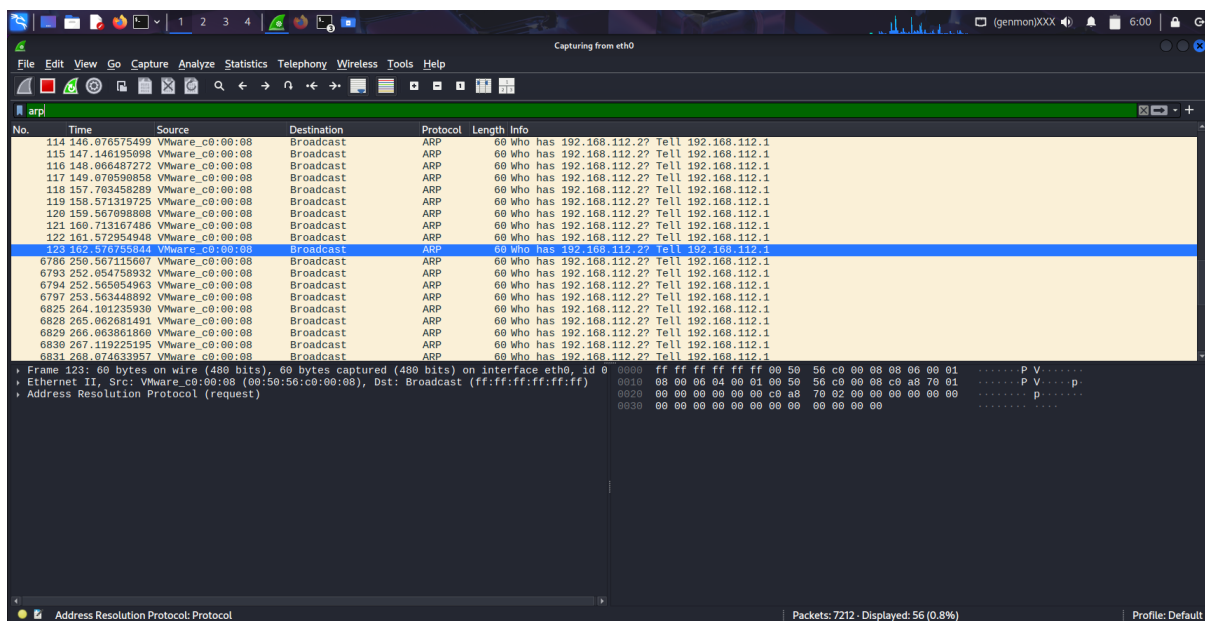
Explanation:

The screenshot shows DNS requests and responses used for translating domain names into IP addresses. DNS traffic plays an important role in establishing communication between systems and websites.

Step 8: ARP Packet Analysis:

The ARP filter was applied to capture Address Resolution Protocol packets. ARP requests were broadcast inside the local network to identify device MAC addresses. This protocol helps systems communicate properly within the same network segment.

Figure 8: ARP Packet Capture:



Explanation:

ARP broadcast requests were captured successfully. These packets are used for resolving IP addresses to MAC addresses inside the local network. ARP communication is important for identifying devices within a local area network.

Observations:

- Elasticsearch service started successfully.
- Kibana web interface opened successfully.
- Live network traffic was captured using Wireshark.
- ICMP packets were generated using the ping command.
- TCP packets showed active communication.
- DNS packets displayed successful domain resolution.
- ARP packets showed address resolution requests.

Conclusion:

This practical task helped in understanding the basics of network traffic monitoring and packet analysis using Wireshark. Different protocols such as ICMP, TCP, DNS, and ARP were captured and analyzed successfully. Elasticsearch and Kibana services were also configured and accessed properly. This task provided practical exposure to SOC monitoring activities and helped improve understanding of real-time network analysis.